Centro de Enseñanza Técnica Industrial
Desarrollo de Software
División de Informática y Computación

Gomitas Dulce Vicio

Métodos de Cobro y Dashboard de Administración

Integración de pagos con Mercado Pago en una API REST con FastAPI y un frontend en React

Materia: Desarrollo Web
Alumno: Diego Maximiliano Sánchez Castro
Registro: 23300652 Grupo: 7F
13 de octubre de 2026

Índice

1. Introducción 3
1.1 Objetivos 3
2. Marco Teórico 3
2.1 Pasarela de pagos y Checkout Pro 3
2.2 Preferencia de pago y referencia externa 3
2.3 Métodos de pago en México 4
2.4 Webhooks y firma HMAC 4
2.5 Indicadores de un e-commerce 4
3. Desarrollo 5
3.1 Archivos nuevos y modificados 5
3.2 Modelo de datos: entidad Pago 5
3.3 Estados del pedido y del pago 6
3.4 Endpoints nuevos 6
3.5 Creación de la preferencia en Mercado Pago 7
3.6 Validación del pago 7
3.7 Webhook con firma 7
3.8 Pasarela simulada 8
3.9 Estadísticas del dashboard 8
4. Evidencia del Funcionamiento 9
4.1 Selección del método de pago 9
4.2 Pago con tarjeta 10
4.3 Efectivo en tienda, link de pago y pago al recoger 11
4.4 Mis pedidos 13
4.5 Dashboard del administrador 13
4.6 Documentación de la API 15
4.7 Pruebas automáticas 15
5. Lo Más Importante que se Implementó 16
5.1 El servidor nunca confía en el navegador 16
5.2 Estado del pedido separado del estado del pago 16
5.3 Un dashboard que responde preguntas del negocio 16
6. Conclusión 16
Anexo. Cómo ejecutar el proyecto 17

1. Introducción

En las prácticas anteriores se construyó el e-commerce de gomitas «Gomitas Dulce Vicio»: un frontend en React que navega por estados (inicio → categoría → carrito → login → checkout) y una API REST en FastAPI con autenticación OAuth2 (password flow con tokens JWT) y autorización por scopes. Hasta ese punto, al confirmar el pedido éste quedaba registrado, pero no existía ningún proceso de cobro.

En esta práctica se agregó el cobro de los pedidos con cinco métodos de pago — tarjeta de crédito, tarjeta de débito, efectivo en tienda de conveniencia (OXXO), link de pago y pago al recoger — integrados con Mercado Pago Checkout Pro, además de un dashboard para que el administrador consulte las estadísticas del negocio: cuántos pagos se hicieron con cada método, cuántos fueron aprobados, rechazados o siguen pendientes, los ingresos por día y los productos más vendidos.

1.1 Objetivos

- Permitir que el cliente elija cómo pagar su pedido y completar el flujo de pago de cada método.
- Integrar la pasarela de pagos Mercado Pago (Checkout Pro) sin confiar en datos que vengan del navegador.
- Separar el estado del pedido del estado del pago, y manejar rechazos, reintentos, cancelaciones y devolución de stock.
- Construir un dashboard de administración con indicadores y gráficas calculados en el servidor.
- Comprobar el funcionamiento con pruebas automáticas y pruebas manuales en el navegador.

2. Marco Teórico

2.1 Pasarela de pagos y Checkout Pro

Una pasarela de pagos es el servicio que se encarga de procesar el cobro con el banco o con la red de pago correspondiente, de modo que la tienda nunca recibe ni almacena los datos de la tarjeta. Mercado Pago ofrece Checkout Pro: la tienda crea una «preferencia de pago» con los productos y el comprador, Mercado Pago responde con un enlace (init_point) y el cliente paga en la página segura de Mercado Pago. Al terminar, el comprador es redirigido de regreso a la tienda.

2.2 Preferencia de pago y referencia externa

La preferencia es un objeto JSON que describe el cobro: **items** (título, cantidad, precio y moneda MXN), **payer** (nombre y correo), **back_urls** (a dónde regresar según el resultado), **payment_methods** (qué tipos de pago se excluyen) y **external_reference**. Esta última es el

identificador del pedido en la tienda; Mercado Pago la devuelve en cada pago, lo que permite saber a qué pedido corresponde un pago y buscar todos los intentos de un mismo pedido.

2.3 Métodos de pago en México

Mercado Pago agrupa los medios de pago por tipo. En este proyecto se usaron los siguientes:

Tipo en Mercado Pago	Ejemplos	Uso en la tienda
`credit_card`	Visa, Mastercard, American Express	Tarjeta de crédito (hasta 12 mensualidades)
`debit_card`	Visa Débito, Mastercard Débito	Tarjeta de débito
`ticket`	OXXO, 7-Eleven y otras tiendas	Efectivo en tienda: se genera una ficha y se paga en caja
Todos	Cualquiera de los anteriores	Link de pago que se puede compartir
— (sin pasarela)	Efectivo o tarjeta en mostrador	Pagar al recoger el pedido en la tienda

Los pagos en efectivo son asíncronos: al generar la ficha el pago queda «pendiente» y se acredita horas o días después, cuando el cliente paga en la tienda. Por eso la tienda necesita una forma de enterarse del cambio de estado sin que el cliente vuelva a entrar al sitio.

2.4 Webhooks y firma HMAC

Un webhook es una petición HTTP que un servicio externo envía al servidor cuando ocurre un evento. Mercado Pago manda un POST con el identificador del pago cada vez que éste cambia. Como cualquiera podría enviar una petición falsa a esa ruta, Mercado Pago firma cada notificación en el encabezado x-signature con HMAC-SHA256, usando una clave secreta compartida. El servidor recalcula la firma y la compara; además, nunca usa los datos del cuerpo de la notificación, sino que vuelve a consultar el pago directamente en la API de Mercado Pago.

2.5 Indicadores de un e-commerce

Un dashboard resume el estado del negocio en indicadores (KPI). Los utilizados fueron: ingresos cobrados, número de pedidos, tasa de aprobación (pagos aprobados entre pagos con resultado final), ticket promedio (ingreso promedio por pago aprobado) y monto por cobrar (pagos pendientes). Para que los datos sean confiables, se calculan en el servidor con consultas de agregación (GROUP BY) sobre la base de datos y no en el navegador.

3. Desarrollo

3.1 Archivos nuevos y modificados

Archivo	Descripción
`back/app/models.py`	Nueva entidad Pago (1:1 con Pedido), enumeraciones MetodoPago, EstadoPago y EstadoPedido.
`back/app/mercadopago.py`	Cliente de la API REST de Mercado Pago con httpx: preferencias, consulta y búsqueda de pagos, validación de la firma del webhook.
`back/app/servicios_pago.py`	Reglas de negocio: iniciar el cobro, aplicar el estado de un pago, cancelar y devolver stock.
`back/app/routers/pagos.py`	Endpoints de pagos: reintentar, verificar, simular, actualización manual y webhook.
`back/app/routers/admin.py`	Endpoint de estadísticas del dashboard.
`back/app/routers/pedidos.py`	El pedido recibe el método de pago; nuevos endpoints de cancelar y entregar; filtros para el administrador.
`back/app/seed.py`	Pedidos de ejemplo de los últimos 30 días para el dashboard.
`back/tests/test_pagos.py`	Pruebas automáticas de pagos, webhook y dashboard.
`front/src/views/PagoView.jsx`	Vista del pago de un pedido según su método.
`front/src/views/MisPedidosView.jsx`	Historial de pedidos del cliente.
`front/src/views/DashboardView.jsx`	Dashboard del administrador.
`front/src/components/Graficas.jsx`	Gráficas hechas con HTML y CSS (barras apiladas, barras simples y columnas).
`db.sql`	Esquema SQL con la nueva tabla pago.

3.2 Modelo de datos: entidad Pago

Se agregó la tabla pago con relación uno a uno con pedido. Se decidió separar el estado del pedido (logística) del estado del pago (dinero): un pedido puede estar «pendiente de pago» mientras su pago fue «rechazado» y el cliente reintenta con otra tarjeta.

Campo	Tipo	Descripción
`id`	int, PK	Identificador
`pedido_id`	int, FK, único	Pedido al que pertenece
`metodo`	enum	tarjeta_credito, tarjeta_debito, oxxo, link_pago, pickup
`estado`	enum	pendiente, aprobado, rechazado, cancelado, reembolsado
`monto`	decimal	Total que se debe cobrar
`proveedor`	texto	mercadopago, simulado o tienda
`mp_preference_id`	texto	Preferencia creada en Mercado Pago
`mp_payment_id`	texto	Identificador del pago en Mercado Pago

Campo	Tipo	Descripción
`mp_tipo / mp_medio`	texto	Tipo y medio reportados (credit_card / visa, ticket / oxxo…)
`url_pago / url_ficha`	texto	Enlace para pagar y enlace de la ficha OXXO
`creado / actualizado`	fecha	Marcas de tiempo

3.3 Estados del pedido y del pago

Evento	Pago	Pedido
Se crea el pedido (el stock se aparta)	pendiente	pendiente_pago
La pasarela rechaza la tarjeta	rechazado	pendiente_pago (puede reintentar)
Se genera la ficha OXXO	pendiente	pendiente_pago
Mercado Pago aprueba el pago	aprobado	pagado
El administrador entrega el pedido	aprobado	entregado
Pickup: se cobra al entregar	aprobado	entregado
Cancelación o ficha vencida (el stock regresa)	cancelado	cancelado
Reembolso	reembolsado	cancelado

3.4 Endpoints nuevos

Método	Ruta	Acceso	Función
POST	`/pedidos`	scope pedidos	Crea el pedido con metodo_pago e inicia el cobro
POST	`/pedidos/{id}/cancelar`	dueño o admin	Cancela y devuelve el stock
POST	`/pedidos/{id}/entregar`	admin	Marca entregado (en pickup también cobra)
GET	`/pedidos?metodo=&estado_pago=`	admin	Lista con filtros
GET	`/pagos/config`	público	Indica si se usa Mercado Pago o la pasarela simulada
POST	`/pagos/{id}/reintentar`	dueño	Nuevo cobro, puede cambiar de método
POST	`/pagos/{id}/verificar`	dueño	Consulta el pago en Mercado Pago
POST	`/pagos/{id}/simular`	dueño	Resultado de la pasarela simulada
PATCH	`/pagos/{id}`	admin	Cambio manual de estado
POST	`/pagos/webhook`	público con firma	Notificaciones de Mercado Pago
GET	`/admin/estadisticas?dias=`	admin	Datos del dashboard

3.5 Creación de la preferencia en Mercado Pago

Para cada método se excluyen los tipos de pago que no corresponden; así, si el cliente eligió «tarjeta de débito», Mercado Pago sólo le muestra tarjetas de débito. El total se calcula en el servidor con los precios de la base de datos:

```
TIPOS_PERMITIDOS = {
    MetodoPago.tarjeta_credito: {"credit_card"},
    MetodoPago.tarjeta_debito: {"debit_card"},
    MetodoPago.oxxo: {"ticket"},          # OXXO y otras tiendas
    MetodoPago.link_pago: set(TIPOS_MP),  # acepta cualquier metodo
}

def crear_preferencia(pedido: Pedido, pago: Pago) -> tuple[str, str]:
    excluidos = [{"id": t} for t in TIPOS_MP if t not in TIPOS_PERMITIDOS[pago.metodo]]
    preferencia = {
        "items": [{"title": d.producto.nombre, "quantity": d.cantidad,
                   "unit_price": d.precio_unitario, "currency_id": "MXN"}
                  for d in pedido.items],
        "payer": {"name": pedido.cliente.nombre, "email": pedido.cliente.email},
        "external_reference": str(pedido.id),
        "back_urls": {"success": regreso, "pending": regreso, "failure": regreso},
        "payment_methods": {"excluded_payment_types": excluidos,
                            "installments": 12 if credito else 1},
    }
    data = _request("POST", "/checkout/preferences", json=preferencia)
    return data["id"], data["init_point"]
```

3.6 Validación del pago

Cuando el cliente regresa de Mercado Pago, la URL sólo indica qué pago revisar. El backend consulta ese pago en la API de Mercado Pago y, antes de aprobar el pedido, valida que pertenezca al pedido y que el monto pagado lo cubra:

```
def aplicar_pago_mp(session, pago: Pago, datos: dict) -> None:
    if str(datos.get("external_reference")) != str(pago.pedido_id):
        raise HTTPException(400, "Ese pago de Mercado Pago es de otro pedido")
    if datos.get("status") == "approved" and datos["transaction_amount"] + 0.01 < pago.monto:
        raise HTTPException(400, "El monto pagado no coincide con el pedido")

    pago.mp_payment_id = str(datos["id"])
    pago.mp_tipo = datos.get("payment_type_id")     # credit_card, ticket...
    pago.mp_medio = datos.get("payment_method_id")  # visa, oxxo...
    estado = ESTADOS_MP.get(datos.get("status"), EstadoPago.pendiente)
    aplicar_estado(session, pago, estado)
```

3.7 Webhook con firma

```
def firma_valida(x_signature, x_request_id, data_id) -> bool:
    partes = dict(p.strip().split("=", 1) for p in x_signature.split(","))
    ts, v1 = partes.get("ts"), partes.get("v1")
    manifest = f"id:{data_id.lower()};request-id:{x_request_id};ts:{ts};"
    esperado = hmac.new(secreto.encode(), manifest.encode(), hashlib.sha256).hexdigest()
    return hmac.compare_digest(esperado, v1)
```

Si la firma no coincide, el endpoint responde 401. Si es válida, vuelve a consultar el pago con obtener_pago(data_id) y aplica el resultado. Se responde 200 aun cuando el pago no se pueda aplicar, para que Mercado Pago no reintente la misma notificación indefinidamente.

3.8 Pasarela simulada

La integración real se activa al configurar la variable MP_ACCESS_TOKEN con las credenciales de la cuenta de Mercado Pago del negocio. Mientras no existan credenciales, el sistema usa una pasarela simulada que reproduce el comportamiento de cada método (tarjeta aprobada o rechazada, generación de ficha OXXO y pago posterior, link de pago) y registra los mismos datos que reportaría Mercado Pago. Esto permite probar el flujo completo y alimentar el dashboard sin realizar cobros reales. La pasarela simulada no solicita datos de tarjeta y se desactiva automáticamente en cuanto se configura Mercado Pago.

3.9 Estadísticas del dashboard

El endpoint /admin/estadisticas calcula los indicadores con consultas agregadas. Por ejemplo, la tabla cruzada de método de pago contra estado del pago:

```
filas = session.exec(
    select(Pago.metodo, Pago.estado, func.count(Pago.id), func.sum(Pago.monto))
    .join(Pedido)
    .where(Pedido.fecha >= desde)
    .group_by(Pago.metodo, Pago.estado)
).all()
```

Con esta información se responde directamente cuántos pagos se hicieron con tarjeta de crédito, cuántos de ellos se aprobaron y cuánto dinero se cobró por cada método. Los ingresos por día se agrupan con la hora local de México (UTC-6), ya que las fechas se guardan en UTC. El periodo se puede filtrar a 7, 30 o 90 días, o a todo el historial.

4. Evidencia del Funcionamiento

4.1 Selección del método de pago

En el checkout el cliente elige uno de los cinco métodos. El botón cambia a «Apartar pedido» cuando se elige pagar al recoger.

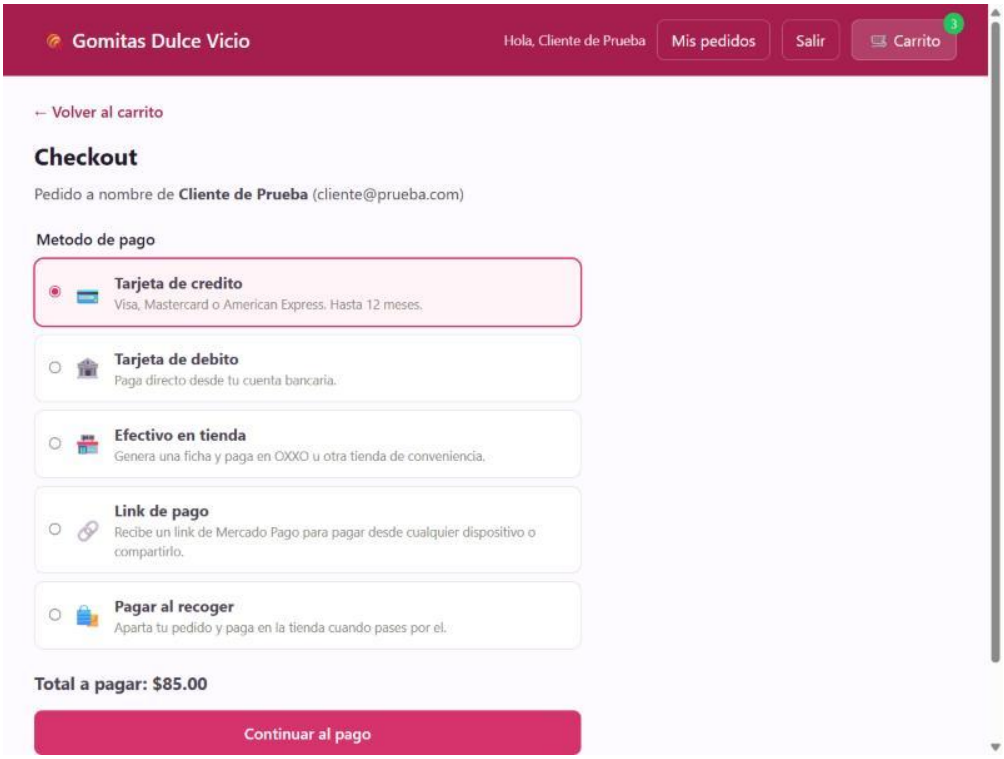


Figura 1. *Checkout con los cinco métodos de pago.*

4.2 Pago con tarjeta

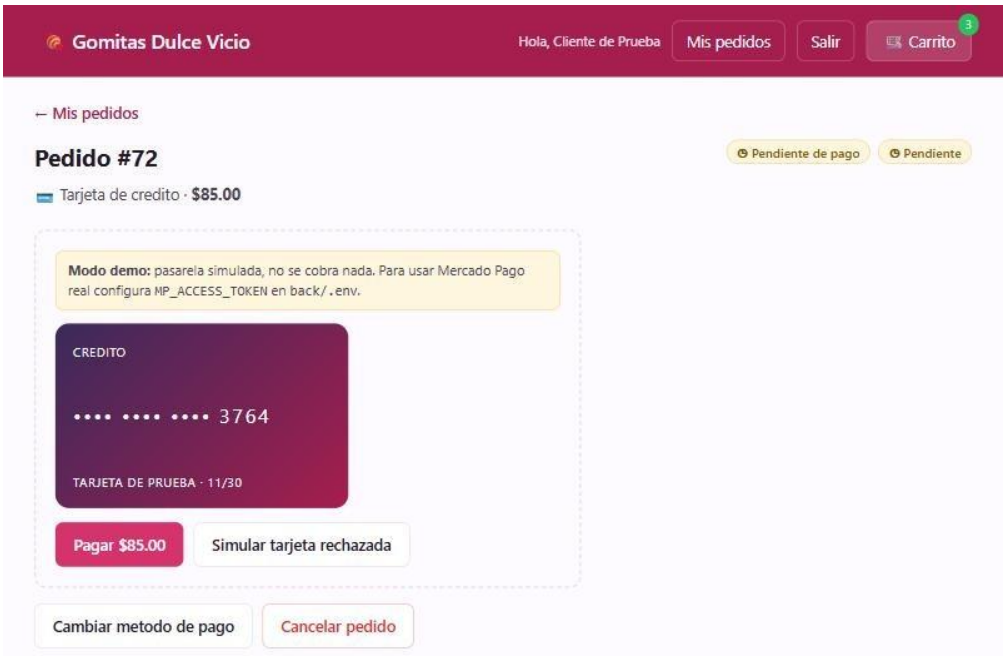


Figura 2. *Pago con tarjeta de crédito (pasarela simulada).*

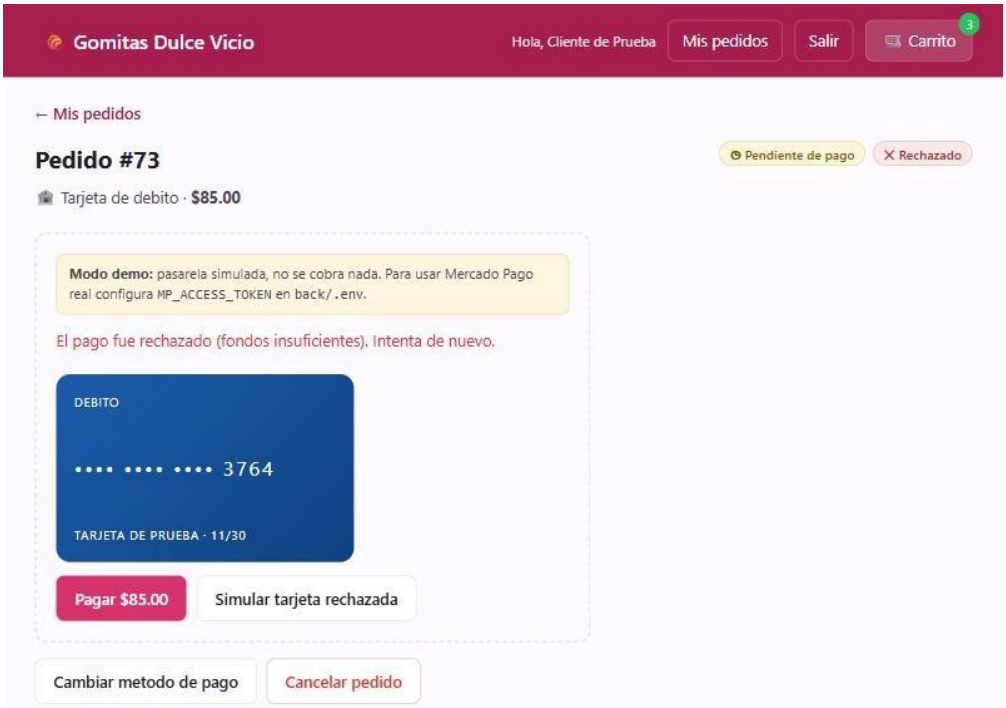


Figura 3. *Tarjeta de débito rechazada: el pedido sigue abierto para reintentar o cambiar de método.*



Figura 4. *Pago aprobado: el pedido pasa a «Pagado» y se registra la marca de la tarjeta.*

4.3 Efectivo en tienda, link de pago y pago al recoger

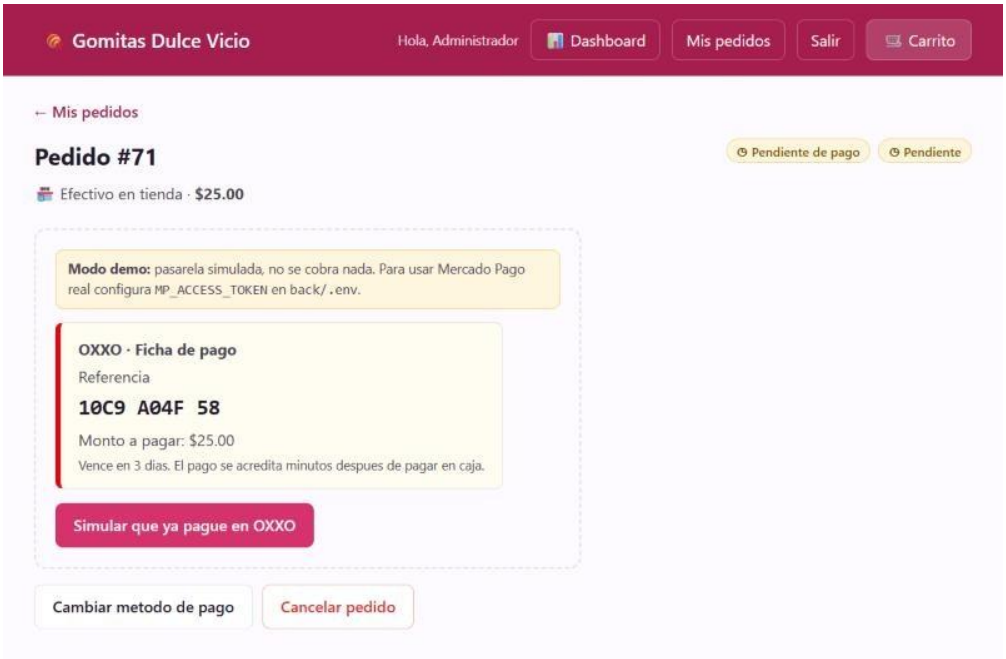


Figura 5. *Ficha de pago para OXXO con referencia y monto; el pago queda pendiente hasta que se paga en caja.*



Figura 6. *Link de pago para compartir.*



Figura 7. *Pedido apartado para pagar al recoger en la tienda.*

4.4 Mis pedidos

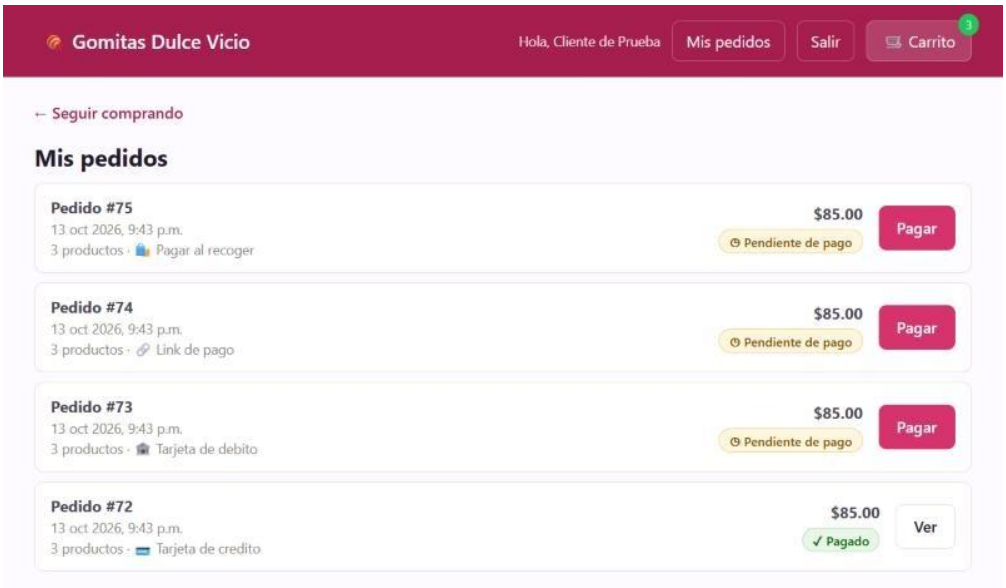


Figura 8. *Historial del cliente con el estado y el método de cada pedido.*

4.5 Dashboard del administrador

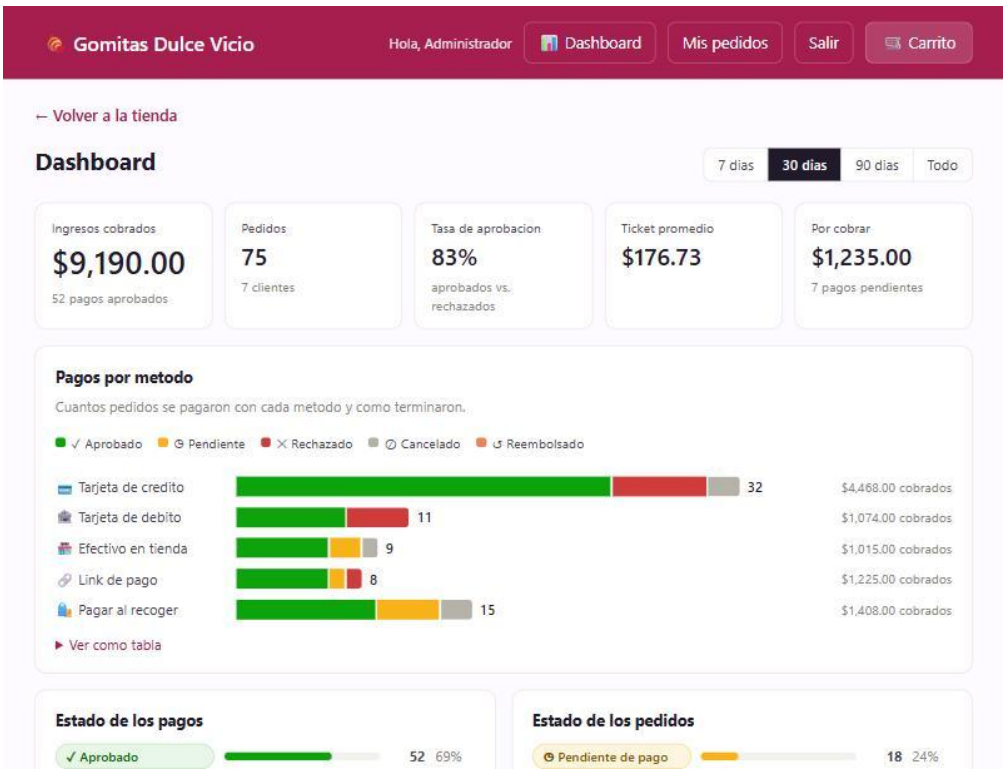


Figura 9. *Indicadores principales y pagos por método, divididos por estado del pago.*



Figura 10. *Ingresos cobrados por día, pagos aprobados por marca y productos más vendidos.*

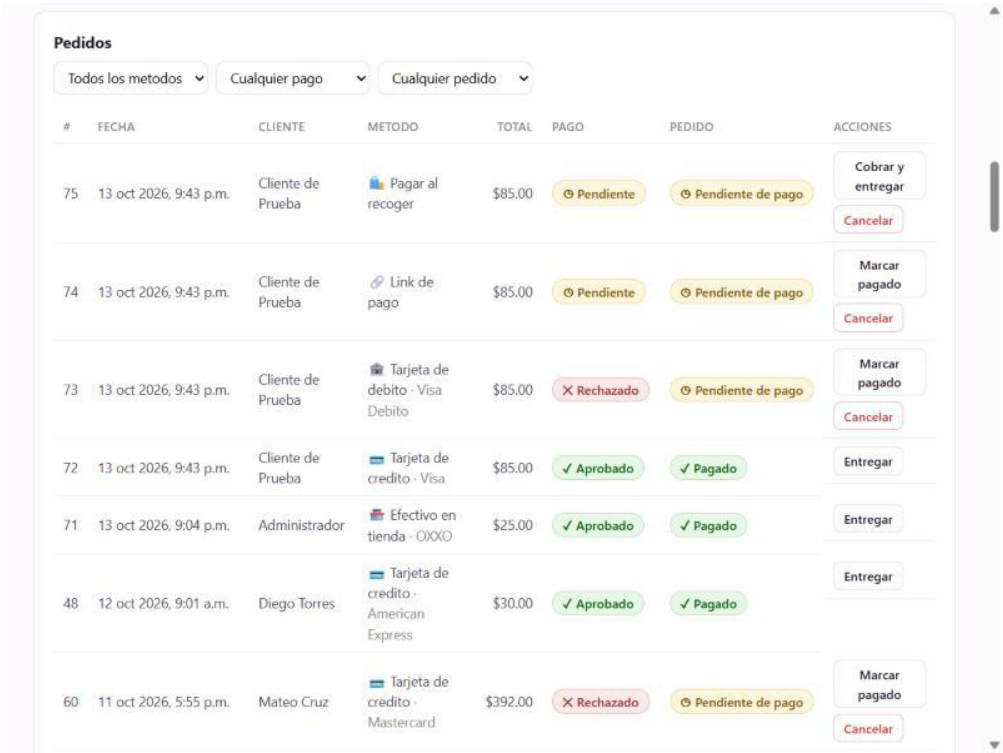


Figura 11. *Tabla de pedidos con filtros y acciones del administrador.*

4.6 Documentación de la API

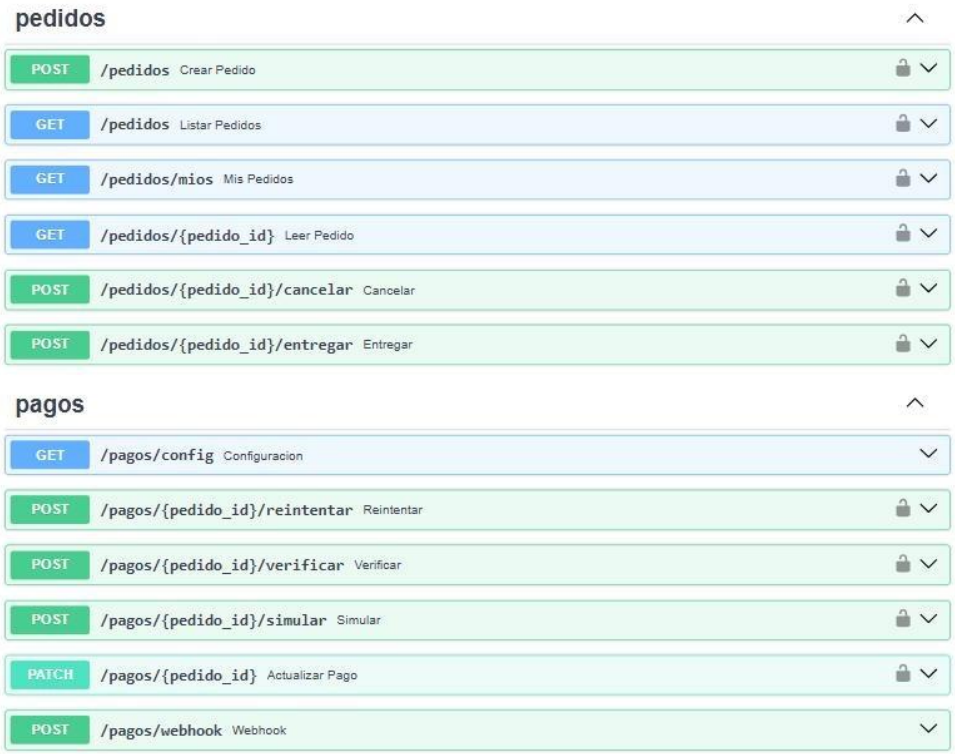


Figura 12. *Endpoints de pedidos y pagos en Swagger UI (/docs).*

4.7 Pruebas automáticas

Las pruebas se ejecutan con pytest sobre una base de datos temporal. Para probar la integración con Mercado Pago sin hacer peticiones reales, las funciones que llaman a su API se reemplazan con monkeypatch por respuestas simuladas con el mismo formato que devuelve Mercado Pago.

```
$ pytest -v
tests/test_api.py::test_flujo_completo PASSED                          [ 33%]
tests/test_pagos.py::test_pasarela_simulada_y_cancelacion PASSED       [ 66%]
tests/test_pagos.py::test_mercadopago_y_webhook PASSED                 [100%]
============================== 3 passed in 4.83s ===============================
```

Casos comprobados:

- Un método de pago inválido responde 422.
- Una tarjeta rechazada y después aprobada deja el pedido en «pagado»; un pedido pagado ya no se puede pagar de nuevo ni cancelar (409).
- Cancelar un pedido sin pagar devuelve el stock de los productos.
- Reintentar cambiando de método; el pago al recoger no pasa por la pasarela.
- El administrador entrega un pedido pickup y en ese momento queda cobrado; un pedido con tarjeta sin pagar no se puede entregar.
- Mercado Pago: ficha OXXO pendiente con su enlace; webhook con firma inválida → 401; webhook válido aprueba el pedido.
- Un pago aprobado por un monto menor, o de otro pedido, se rechaza con 400.

- Con varios intentos (uno rechazado y otro aprobado) se conserva el aprobado.
- Un cliente no puede consultar el dashboard (403); el administrador recibe los conteos por método y estado, las ventas de 30 días y los productos más vendidos.

5. Lo Más Importante que se Implementó

5.1 El servidor nunca confía en el navegador

El resultado del pago no se toma de la URL de regreso ni del cuerpo del webhook: siempre se consulta en la API de Mercado Pago con el token privado de la tienda, y se valida que el pago corresponda al pedido y cubra el total. El precio de cada producto también se toma de la base de datos y no del carrito del cliente.

5.2 Estado del pedido separado del estado del pago

Tener dos estados independientes permitió modelar casos reales: reintentar una tarjeta rechazada sin crear otro pedido, cobrar un pickup al momento de entregarlo, o registrar un pago en OXXO que llega después de que el pedido fue cancelado para que se reembolse.

5.3 Un dashboard que responde preguntas del negocio

Las gráficas muestran cada método de pago dividido por resultado, por lo que de un vistazo se sabe qué métodos generan más rechazos o cuánto dinero está pendiente de cobrar. Los colores de estado siempre van acompañados de ícono y texto, y cada gráfica tiene leyenda, detalle al pasar el mouse y una vista en tabla.

6. Conclusión

Integrar una pasarela de pagos mostró que el cobro no es un solo paso: hay pagos inmediatos (tarjetas), pagos diferidos (efectivo en tienda) y pagos fuera de línea (al recoger), y cada uno necesita su propio flujo de estados. El uso de external_reference, la consulta directa a la API de Mercado Pago y la validación de la firma del webhook garantizan que un pedido sólo se marque como pagado cuando el dinero realmente se cobró.

La pasarela simulada permitió desarrollar y probar todo el sistema sin depender de credenciales externas, y el diseño deja la integración real lista para activarse con la cuenta del negocio. Finalmente, el dashboard convierte los datos de pedidos y pagos en información útil para el administrador, calculada en el servidor con consultas agregadas.

Anexo. Cómo ejecutar el proyecto

1. Requisitos: Python 3.10 o superior y Node.js 20 o superior.
2. En Windows, ejecutar **INICIAR.bat**: instala dependencias, arranca el backend (http://localhost:8000) y el frontend (http://localhost:5173) y abre el navegador.
3. Ejecutar **PROBAR.bat** para correr las pruebas automáticas.
4. Usuarios de prueba: administrador admin@gomitas.com / admin12345 y cliente cliente@prueba.com / cliente12345.
5. Para usar Mercado Pago real, colocar el Access Token en back/.env (MP_ACCESS_TOKEN) y reiniciar el backend.